figs/logo.png

Recursão

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br

[1]Faculdade de Sistemas de Informação - UFPA/CUTINS

25 de março de 2026

Roteiro

figs/logo.png

Objetivos

- **Propósito geral:** Apresentar a recursividade e sua potencialidade na resolução de problemas complexos na Ciência da Computação.
- Ao final desta aula, o aluno será capaz de:
  - Definir rigorosamente os conceitos de **caso base** e **passo indutivo** em algoritmos.
  - Distinguir cenários ideais para aplicar a estratégia recursiva em problemas práticos.
  - Formular relações de **recorrência** para delimitar complexidade (Abordagem de Divisão e Conquista).
  - Compreender desafios reais, incluindo recursões implícitas e problemas relativos à pilha de execução (Stack Overflow).

figs/logo.png

Objetivos

- **Propósito geral:** Apresentar a recursividade e sua potencialidade na resolução de problemas complexos na Ciência da Computação.
- Ao final desta aula, o aluno será capaz de:
  - Definir rigorosamente os conceitos de **caso base** e **passo indutivo** em algoritmos.
  - Distinguir cenários ideais para aplicar a estratégia recursiva em problemas práticos.
  - Formular relações de **recorrência** para delimitar complexidade (Abordagem de Divisão e Conquista).
  - Compreender desafios reais, incluindo recursões implícitas e problemas relativos à pilha de execução (Stack Overflow).

figs/logo.png

Objetivos

- **Propósito geral:** Apresentar a recursividade e sua potencialidade na resolução de problemas complexos na Ciência da Computação.
- Ao final desta aula, o aluno será capaz de:
  - Definir rigorosamente os conceitos de **caso base** e **passo indutivo** em algoritmos.
  - Distinguir cenários ideais para aplicar a estratégia recursiva em problemas práticos.
  - Formular relações de **recorrência** para delimitar complexidade (Abordagem de Divisão e Conquista).
  - Compreender desafios reais, incluindo recursões implícitas e problemas relativos à pilha de execução (Stack Overflow).

figs/logo.png

Objetivos

- **Propósito geral:** Apresentar a recursividade e sua potencialidade na resolução de problemas complexos na Ciência da Computação.
- Ao final desta aula, o aluno será capaz de:
  - Definir rigorosamente os conceitos de **caso base** e **passo indutivo** em algoritmos.
  - Distinguir cenários ideais para aplicar a estratégia recursiva em problemas práticos.
  - Formular relações de **recorrência** para delimitar complexidade (Abordagem de Divisão e Conquista).
  - Compreender desafios reais, incluindo recursões implícitas e problemas relativos à pilha de execução (Stack Overflow).

figs/logo.png

Objetivos

- **Propósito geral:** Apresentar a recursividade e sua potencialidade na resolução de problemas complexos na Ciência da Computação.
- Ao final desta aula, o aluno será capaz de:
  - Definir rigorosamente os conceitos de **caso base** e **passo indutivo** em algoritmos.
  - Distinguir cenários ideais para aplicar a estratégia recursiva em problemas práticos.
  - Formular relações de **recorrência** para delimitar complexidade (Abordagem de Divisão e Conquista).
  - Compreender desafios reais, incluindo recursões implícitas e problemas relativos à pilha de execução (Stack Overflow).

figs/logo.png

O que é Recursão?

Na Matemática

Consiste em definir um objeto matemático (função, sequência ou conjunto) em termos da sua própria definição preestabelecida de forma encapsulada.

Na Computação

É o mecanismo pelo qual uma sub-rotina (função, método procedimental) invoca intencionalmente a **si mesma** visando solucionar subproblemas progressivamente menores, até satisfazer uma condição trivial preexistente.

O que é Recursão?

Na Matemática

Consiste em definir um objeto matemático (função, sequência ou conjunto) em termos da sua própria definição preestabelecida de forma encapsulada.

Na Computação

É o mecanismo pelo qual uma sub-rotina (função, método procedimental) invoca intencionalmente a **si mesma** visando solucionar subproblemas progressivamente menores, até satisfazer uma condição trivial preexistente.

figs/logo.png

O que é Recursão?

Na Matemática

Consiste em definir um objeto matemático (função, sequência ou conjunto) em termos da sua própria definição preestabelecida de forma encapsulada.

Na Computação

É o mecanismo pelo qual uma sub-rotina (função, método procedimental) invoca intencionalmente a **si mesma** visando solucionar subproblemas progressivamente menores, até satisfazer uma condição trivial preexistente.

figs/logo.png

Pilares da Recursividade em Algoritmos

O avanço para a resolução de sistemas recursivos repousa necessariamente em duas bases axiomáticas:

1. Caso Base (Condição de Ancora)

Demarcação onde o limite da rotina é alcançado. Quando atingida, emite uma resposta computável prontamente (convergência real), assegurando encerramento.

2. Passo Indutivo

Caminho analítico reduzido de instâncias macro. A cada passo, o algoritmo gera dados que tendem iterativamente até encontrar o caso base.

/logo.png

Pilares da Recursividade em Algoritmos

O avanço para a resolução de sistemas recursivos repousa necessariamente em duas bases axiomáticas:

1. Caso Base (Condição de Ancora)

Demarcação onde o limite da rotina é alcançado. Quando atingida, emite uma resposta computável prontamente (convergência real), assegurando encerramento.

2. Passo Indutivo

Caminho analítico reduzido de instâncias macro. A cada passo, o algoritmo gera dados que tendem iterativamente até encontrar o caso base.

/logo.png

Pilares da Recursividade em Algoritmos

O avanço para a resolução de sistemas recursivos repousa necessariamente em duas bases axiomáticas:

1. Caso Base (Condição de Ancora)

Demarcação onde o limite da rotina é alcançado. Quando atingida, emite uma resposta computável prontamente (convergência real), assegurando encerramento.

2. Passo Indutivo

Caminho analítico reduzido de instâncias macro. A cada passo, o algoritmo gera dados que tendem iterativamente até encontrar o caso base.

/logo.png

Exercício Lógico: Fatorial Matemático ($n!$)

Para dominar recursividade escalar, extraímos o primórdio produto de cadeias:

Definição Recursiva Equacional

$$n! = \begin{cases} 1, & \text{se } n = 0 \quad \textbf{(Caso Base)} \\ n \cdot (n-1)!, & \text{se } n > 0 \quad \textbf{(Passo de Indução)} \end{cases}$$

Implementação Computacional:

```
def fatorial(n):
    if n == 0:
        return 1
    else:
        return n * fatorial(n - 1)
```

figs/logo.png

Exercício Lógico: Fatorial Matemático ($n!$)

Para dominar recursividade escalar, extraímos o primórdio produto de cadeias:

Definição Recursiva Equacional

$$n! = \begin{cases} 1, & \text{se } n = 0 \quad \textbf{(Caso Base)} \\ n \cdot (n-1)!, & \text{se } n > 0 \quad \textbf{(Passo de Indução)} \end{cases}$$

Implementação Computacional:

```
def fatorial(n):
    if n == 0:
        return 1
    else:
        return n * fatorial(n - 1)
```

figs/logo.png

Tratamento de Contexto: Pilha de Execução (*Call Stack*)

Como o sistema preserva variáveis locais entre invocações? Todo compilador provisiona uma **Pilha (Stack)** para estruturar sucessões não resolvidas.

figs/logo.png

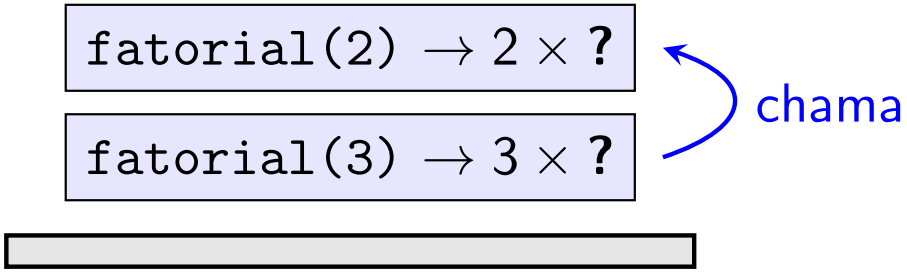
Tratamento de Contexto: Pilha de Execução (*Call Stack*)

Como o sistema preserva variáveis locais entre invocações? Todo compilador provisiona uma **Pilha (Stack)** para estruturar sucessões não resolvidas.

`fatorial(3)` $\rightarrow 3 \times$ **?**

Tratamento de Contexto: Pilha de Execução (*Call Stack*)

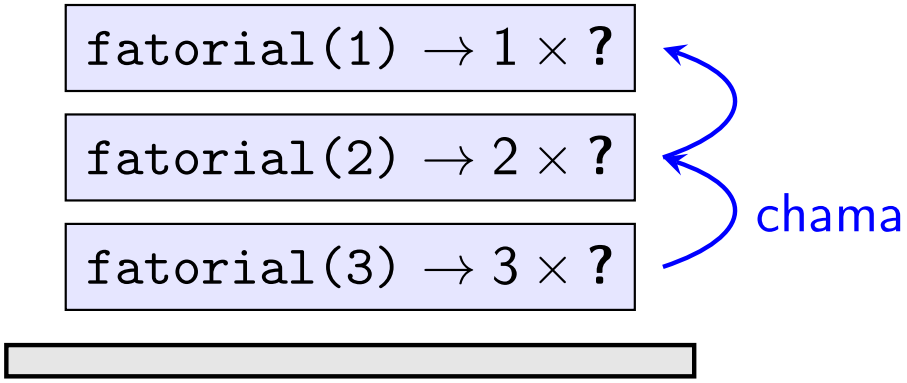
Como o sistema preserva variáveis locais entre invocações? Todo compilador provisiona uma **Pilha (Stack)** para estruturar sucessões não resolvidas.



figs/logo.png

Tratamento de Contexto: Pilha de Execução (*Call Stack*)

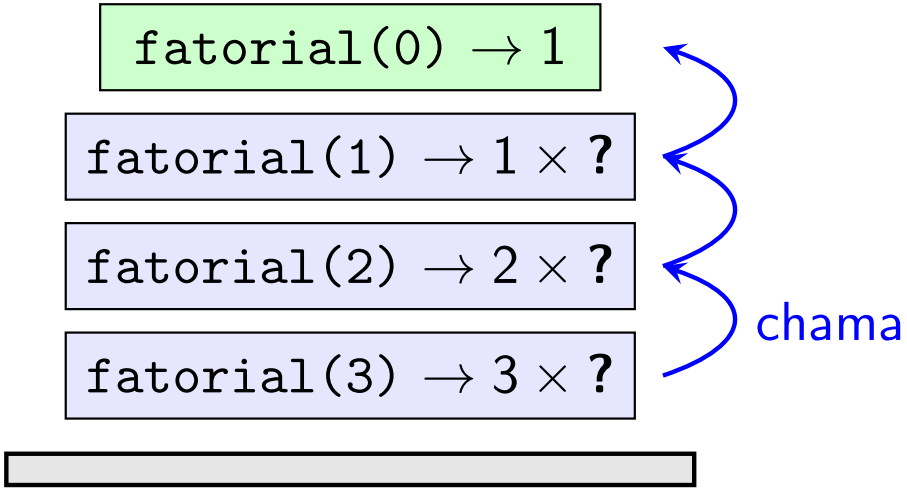
Como o sistema preserva variáveis locais entre invocações? Todo compilador provisiona uma **Pilha (Stack)** para estruturar sucessões não resolvidas.



figs/logo.png

Tratamento de Contexto: Pilha de Execução (*Call Stack*)

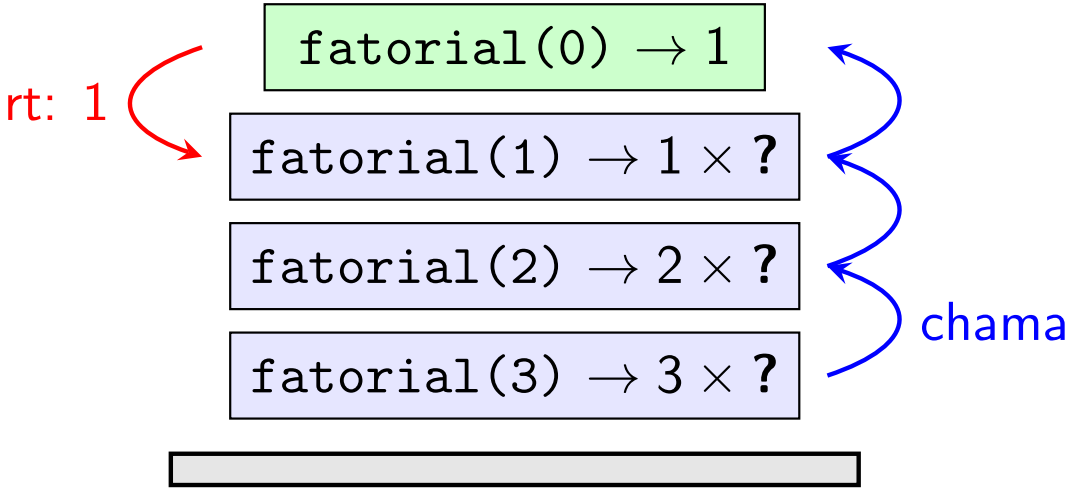
Como o sistema preserva variáveis locais entre invocações? Todo compilador provisiona uma **Pilha (Stack)** para estruturar sucessões não resolvidas.



figs/logo.png

Tratamento de Contexto: Pilha de Execução (*Call Stack*)

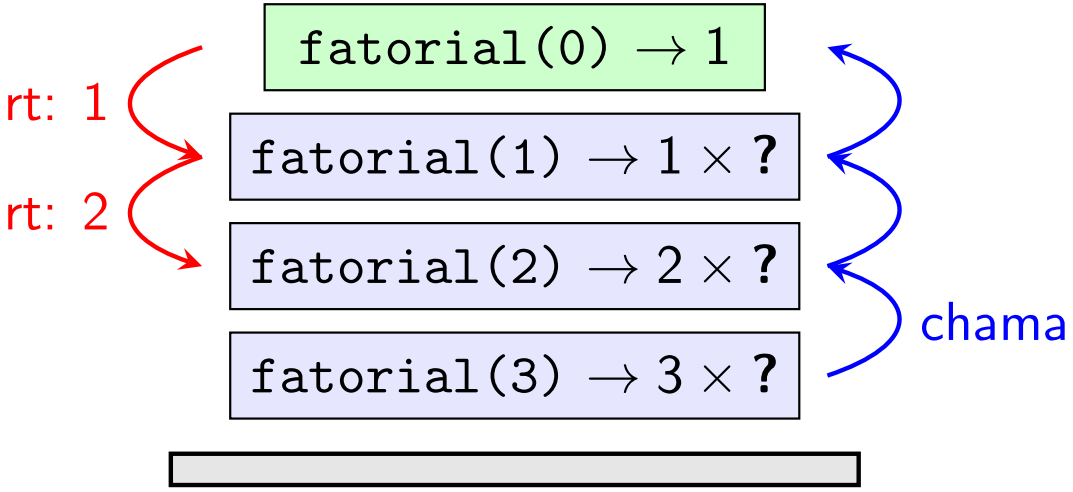
Como o sistema preserva variáveis locais entre invocações? Todo compilador provisiona uma **Pilha (Stack)** para estruturar sucessões não resolvidas.



figs/logo.png

Tratamento de Contexto: Pilha de Execução (*Call Stack*)

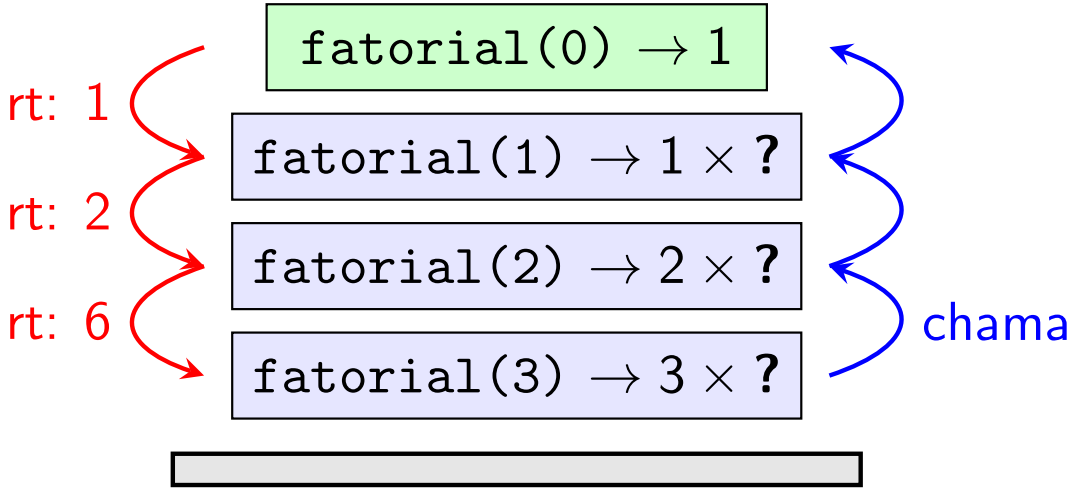
Como o sistema preserva variáveis locais entre invocações? Todo compilador provisiona uma **Pilha (Stack)** para estruturar sucessões não resolvidas.



figs/logo.png

Tratamento de Contexto: Pilha de Execução (*Call Stack*)

Como o sistema preserva variáveis locais entre invocações? Todo compilador provisiona uma **Pilha (Stack)** para estruturar sucessões não resolvidas.



figs/logo.png

Múltipla Ramificação: Sequência de Fibonacci

Cálculos recursivos podem propagar expansões laterais massivas. Fibonacci exemplifica ramificação de profundidade.

Relação de Recorrência

$$f(n) = \begin{cases} 0, & \text{se } n = 0 \quad \textbf{(Base A)} \\ 1, & \text{se } n = 1 \quad \textbf{(Base B)} \\ f(n-1) + f(n-2), & \text{se } n > 1 \quad \textbf{(Bifurcação de Passo)} \end{cases}$$

- Termos avaliados ($f(0)$ em diante):

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55 \ldots$$

figs/logo.png

Múltipla Ramificação: Sequência de Fibonacci

Cálculos recursivos podem propagar expansões laterais massivas. Fibonacci exemplifica ramificação de profundidade.

Relação de Recorrência

$$f(n) = \begin{cases} 0, & \text{se } n = 0 \quad (\textbf{Base A}) \\ 1, & \text{se } n = 1 \quad (\textbf{Base B}) \\ f(n-1) + f(n-2), & \text{se } n > 1 \quad (\textbf{Bifurcação de Passo}) \end{cases}$$

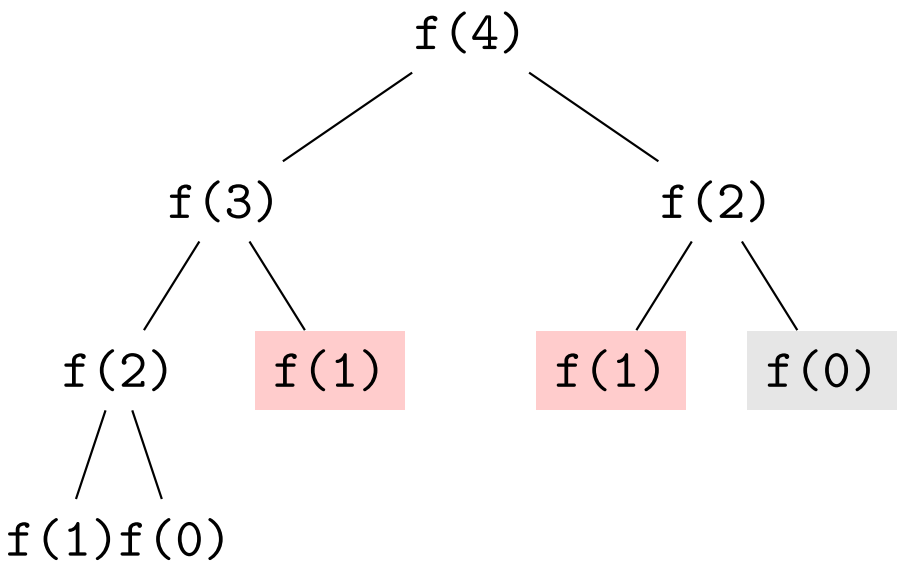
- Termos avaliados ($f(0)$ em diante):

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55 \ldots$$

figs/logo.png

Desdobramento Assintótico: Árvore Fibonacci

Demonstrativo do Overlap (Superposição) Computacional $\Longrightarrow O(2^n)$.



figs/logo.png

Avaliação Qualitativa: Omissões e Otimizações

- **Recursos de Cauda (Tail Recursion):** Se a etapa induzida representar fundamentalmente a encerramento natural avaliado pelo ciclo isolado, sistemas contemporâneos convertem nativamente sob $O(1)$ *em memórias* contornando a pilha.
- **Vícios Axiomáticos:** Abertura recursiva com condicionalidade submetida incorretamente resulta por si em loop infinito.

figs/logo.png

Trabalho Avaliativo - Domínio Modular I

1. **Múltiplo Escalamento Marginal:** Implemente uma diretriz computacional rigorosamente recursiva visando recuperar sem iteração paralela, o mínimo absoluto (menor elemento) alocado sobre um grande vetor desornado.
2. **Segmentações Restritas:** Formule uma assinatura capaz de renderizar perfeitamente todas as partições (*substrings contíguas*) provenientes de primitivas sequenciais puras. Exemplo isolado à constante $S \equiv$ dog:

$$|d|\ do\ |\ dog\ |\ o\ |\ og\ |\ g|$$

figs/logo.png

Trabalho Avaliativo - Domínio Modular II

3. **Manipulação Indutiva Discreta:** Analise a sucessão processual subjacente a representação $g_1 \ldots g_n$ perante um parâmetro demarcador universal k:

$$g_j = \begin{cases} j - 1 & \textbf{se} \;\; 1 \leq j \leq k \\ g_{j-1} + g_{j-2} & \textbf{se} \;\; j > k \end{cases} \tag{1}$$

Elabore um algoritmo logico com base nessa expressão matemática.

figs/logo.png

Compreendendo Recorrências em Algoritmos

A teoria analítica repousa fortemente no dimensionamento explícito de processos autodescritos via formulações restritivas.

Formalização Clássica

Consistem de formulações matemáticas parametrizadas regidas sobre a entrada que especificam **tempo ou recursos limitantes** expressos nos domínios marginais subsequentes da própria recursão:

$$T(n) = T(n-1) + \Theta(1)$$

- **Presença Essencial:** Predominam na validação métrica da tríade recursiva (Dividir, Resolver, Combinar) como MergeSort.
- **Abordagens Propostas:** Visam isolar matematicamente expressões absolutas $\Theta(\dots)$ eliminando chamadas cíclicas relativas ao próprio $T()$.

figs/logo.png

Compreendendo Recorrências em Algoritmos

A teoria analítica repousa fortemente no dimensionamento explícito de processos autodescritos via formulações restritivas.

Formalização Clássica

Consistem de formulações matemáticas parametrizadas regidas sobre a entrada que especificam **tempo ou recursos limitantes** expressos nos domínios marginais subsequentes da própria recursão:

$$T(n) = T(n-1) + \Theta(1)$$

- **Presença Essencial:** Predominam na validação métrica da tríade recursiva (Dividir, Resolver, Combinar) como MergeSort.
- **Abordagens Propostas:** Visam isolar matematicamente expressões absolutas $\Theta(\dots)$ eliminando chamadas cíclicas relativas ao próprio *T()*.

figs/logo.png

Gabaritos Universais: Classificadores Clássicos

Revisitando matrizes temporais padrão de mercado na Ciência Computacional Básica:

$T(n) = T(n-1) + \Theta(1) \implies O(n)$

Extração serial exaure apenas passo elementar (Buscas desordenadas).

$T(n) = T(n-1) + n \implies O(n^2)$

Varreduras residuais lineares repetidas perante extração paralela fracionada.

$T(n) = T(n/2) + c \implies \Theta(\log n)$

Segmentação simétrica com triagem binária restrita constante (Buscadores Diretos).

$T(n) = 2T(n/2) + n \implies \Theta(n \log n)$

Ramificações paralelas onde as fusões custam um empenho linear absoluto sobre o nível arbóreo correspondente.

figs/logo.png

Gabaritos Universais: Classificadores Clássicos

Revisitando matrizes temporais padrão de mercado na Ciência Computacional Básica:

$T(n) = T(n-1) + \Theta(1) \implies O(n)$

Extração serial exaure apenas passo elementar (Buscas desordenadas).

$T(n) = T(n-1) + n \implies O(n^2)$

Varreduras residuais lineares repetidas perante extração paralela fracionada.

$T(n) = T(n/2) + c \implies \Theta(\log n)$

Segmentação simétrica com triagem binária restrita constante (Buscadores Diretos).

$T(n) = 2T(n/2) + n \implies \Theta(n \log n)$

Ramificações paralelas onde as fusões custam um empenho linear absoluto sobre o nível arbóreo correspondente.

figs/logo.png

Gabaritos Universais: Classificadores Clássicos

Revisitando matrizes temporais padrão de mercado na Ciência Computacional Básica:

$T(n) = T(n-1) + \Theta(1) \implies O(n)$

Extração serial exaure apenas passo elementar (Buscas desordenadas).

$T(n) = T(n-1) + n \implies O(n^2)$

Varreduras residuais lineares repetidas perante extração paralela fracionada.

$T(n) = T(n/2) + c \implies \Theta(\log n)$

Segmentação simétrica com triagem binária restrita constante (Buscadores Diretos).

$T(n) = 2T(n/2) + n \implies \Theta(n \log n)$

Ramificações paralelas onde as fusões custam um empenho linear absoluto sobre o nível arbóreo correspondente.

figs/logo.png

Gabaritos Universais: Classificadores Clássicos

Revisitando matrizes temporais padrão de mercado na Ciência Computacional Básica:

$T(n) = T(n-1) + \Theta(1) \implies O(n)$

Extração serial exaure apenas passo elementar (Buscas desordenadas).

$T(n) = T(n-1) + n \implies O(n^2)$

Varreduras residuais lineares repetidas perante extração paralela fracionada.

$T(n) = T(n/2) + c \implies \Theta(\log n)$

Segmentação simétrica com triagem binária restrita constante (Buscadores Diretos).

$T(n) = 2T(n/2) + n \implies \Theta(n \log n)$

Ramificações paralelas onde as fusões custam um empenho linear absoluto sobre o nível arbóreo correspondente.

figs/logo.png

Gabaritos Universais: Classificadores Clássicos

Revisitando matrizes temporais padrão de mercado na Ciência Computacional Básica:

$T(n) = T(n-1) + \Theta(1) \implies O(n)$

Extração serial exaure apenas passo elementar (Buscas desordenadas).

$T(n) = T(n-1) + n \implies O(n^2)$

Varreduras residuais lineares repetidas perante extração paralela fracionada.

$T(n) = T(n/2) + c \implies \Theta(\log n)$

Segmentação simétrica com triagem binária restrita constante (Buscadores Diretos).

$T(n) = 2T(n/2) + n \implies \Theta(n \log n)$

Ramificações paralelas onde as fusões custam um empenho linear absoluto sobre o nível arbóreo correspondente.

figs/logo.png

Método Avançado por Substituição (Desdobramento)

Baseia-se no colapso paramétrico gradual almejando identificar séries progressivas subjacentes. Assumindo modelo primário: $\mathbf{T(n) = T\left(\frac{n}{2}\right) + c}$

Expansão Passo-a-Passo:

$$T(n) = \mathbf{T(n/2)} + c = [\mathbf{T(n/4)} + \mathbf{c}] + c$$

$$= \mathbf{T(n/2^2)} + \mathbf{2c}$$

$$= \mathbf{T(n/2^3)} + \mathbf{3c} = \mathbf{T(n/2^4)} + \mathbf{4c}$$

$$\mathbf{T(n/2^k)} + \mathbf{kc} = \mathbf{T(1)} + \mathbf{c\log_2 n}$$

figs/logo.png

Método Avançado por Substituição (Desdobramento)

Baseia-se no colapso paramétrico gradual almejando identificar séries progressivas subjacentes. Assumindo modelo primário: $\mathbf{T(n) = T\left(\frac{n}{2}\right) + c}$

Expansão Passo-a-Passo:

$$
\begin{aligned}
T(n) &= \mathbf{T(n/2)} + c = [\mathbf{T(n/4) + c}] + c \\
&= \mathbf{T(n/2^2) + 2c} \\
&= [\mathbf{T(n/8) + c}] + 2c = \mathbf{T(n/2^3) + 3c} \\
&\Longrightarrow \cdots \\
&= \mathbf{T(n/2^k) + c \cdot k} \quad \text{(Equação Genérica de pulso } k\text{)}
\end{aligned}
$$

figs/logo.png

Método Avançado por Substituição (Desdobramento)

Baseia-se no colapso paramétrico gradual almejando identificar séries progressivas subjacentes. Assumindo modelo primário: $\mathbf{T(n) = T\left(\frac{n}{2}\right) + c}$

Expansão Passo-a-Passo:

$$
\begin{aligned}
T(n) &= \mathbf{T(n/2)} + c = [\mathbf{T(n/4) + c}] + c \\
&= \mathbf{T(n/2^2) + 2c} \\
&= [\mathbf{T(n/8) + c}] + 2c = \mathbf{T(n/2^3) + 3c} \\
&\Longrightarrow \cdots \\
&= \mathbf{T(n/2^k) + c \cdot k} \quad \text{(Equação Genérica de pulso } k\text{)}
\end{aligned}
$$

figs/logo.png

Método Avançado por Substituição (Desdobramento)

Baseia-se no colapso paramétrico gradual almejando identificar séries progressivas subjacentes. Assumindo modelo primário: $\mathbf{T(n) = T\left(\frac{n}{2}\right) + c}$

Expansão Passo-a-Passo:

$$
\begin{aligned}
T(n) &= \mathbf{T(n/2)} + c = [\mathbf{T(n/4) + c}] + c \\
&= \mathbf{T(n/2^2) + 2c} \\
&= [\mathbf{T(n/8) + c}] + 2c = \mathbf{T(n/2^3) + 3c} \\
&\Longrightarrow \cdots \\
&= \mathbf{T(n/2^k) + c \cdot k} \quad \text{(Equação Genérica de pulso } k\text{)}
\end{aligned}
$$

figs/logo.png

Método Avançado por Substituição (Desdobramento)

Baseia-se no colapso paramétrico gradual almejando identificar séries progressivas subjacentes. Assumindo modelo primário: $\mathbf{T(n) = T\left(\frac{n}{2}\right) + c}$

Expansão Passo-a-Passo:

$$
\begin{aligned}
T(n) &= \mathbf{T(n/2)} + c = [\mathbf{T(n/4) + c}] + c \\
&= \mathbf{T(n/2^2) + 2c} \\
&= [\mathbf{T(n/8) + c}] + 2c = \mathbf{T(n/2^3) + 3c} \\
&\implies \cdots \\
&= \mathbf{T(n/2^k) + c \cdot k} \quad \text{(Equação Genérica de pulso } k\text{)}
\end{aligned}
$$

figs/logo.png

Método Avançado por Substituição (Desdobramento)

Baseia-se no colapso paramétrico gradual almejando identificar séries progressivas subjacentes. Assumindo modelo primário: $\mathbf{T(n) = T\left(\frac{n}{2}\right) + c}$

Expansão Passo-a-Passo:

$$
\begin{aligned}
T(n) &= \mathbf{T(n/2)} + c = [\mathbf{T(n/4) + c}] + c \\
&= \mathbf{T(n/2^2) + 2c} \\
&= [\mathbf{T(n/8) + c}] + 2c = \mathbf{T(n/2^3) + 3c} \\
&\implies \ldots \\
&= \mathbf{T(n/2^k) + c \cdot k} \quad \text{(Equação Genérica de pulso } k\text{)}
\end{aligned}
$$

figs/logo.png

Método Avançado por Substituição (Desdobramento)

Baseia-se no colapso paramétrico gradual almejando identificar séries progressivas subjacentes. Assumindo modelo primário: $\mathbf{T(n) = T\left(\frac{n}{2}\right) + c}$

Expansão Passo-a-Passo:

$$
\begin{aligned}
T(n) &= \mathbf{T(n/2)} + c = [\mathbf{T(n/4) + c}] + c \\
&= \mathbf{T(n/2^2) + 2c} \\
&= [\mathbf{T(n/8) + c}] + 2c = \mathbf{T(n/2^3) + 3c} \\
&\implies \ldots \\
&= \mathbf{T(n/2^k) + c \cdot k} \quad \text{(Equação Genérica de pulso } k\text{)}
\end{aligned}
$$

figs/logo.png

Isolamento da Variável k em Escala Base

Assumindo que nossa âncora de saída (Condição Base) processa arranjos iteracionais triviais de tamanho 1, inferimos $\mathbf{T(1) = 1}$.

Buscando convergência $n/2^k = 1$

$$\frac{n}{2^k} = 1 \implies n = 2^k$$

$$\implies \log_2(n) = \log_2(2^k)$$

$$\implies k = \log_2 n$$

Retornando o valor k, finalizamos para as iterações da Pesquisa:

$$T(n) = T(1) + c \cdot (\log_2 n)$$

figs/logo.png

Conclusão Assintótica

O Algoritmo opera quantitativamente por custo marginal rigoroso $\Theta(\log \mathbf{n})$.

Isolamento da Variável k em Escala Base

Assumindo que nossa âncora de saída (Condição Base) processa arranjos iteracionais triviais de tamanho 1, inferimos $\mathbf{T(1) = 1}$.

Buscando convergência $n/2^k = 1$

$$\begin{aligned} \frac{n}{2^k} = 1 \quad &\Longrightarrow \quad n = 2^k \\ &\Longrightarrow \quad \log_2(n) = \log_2(2^k) \\ &\Longrightarrow \quad k = \log_2 n \end{aligned}$$

Retornando o valor k, finalizamos para as iterações da Pesquisa:

$$T(n) = T(1) + c \cdot (\log_2 n)$$

figs/logo.png

Conclusão Assintótica

O Algoritmo opera quantitativamente por custo marginal rigoroso $\mathbf{\Theta(\log n)}$.

Isolamento da Variável k em Escala Base

Assumindo que nossa âncora de saída (Condição Base) processa arranjos iteracionais triviais de tamanho 1, inferimos $\mathbf{T(1) = 1}$.

Buscando convergência $n/2^k = 1$

$$\frac{n}{2^k} = 1 \implies n = 2^k$$
$$\implies \log_2(n) = \log_2(2^k)$$
$$\implies k = \log_2 n$$

Retornando o valor k, finalizamos para as iterações da Pesquisa:

$$T(n) = T(1) + c \cdot (\log_2 n)$$

figs/logo.png

Conclusão Assintótica

O Algoritmo opera quantitativamente por custo marginal rigoroso $\mathbf{\Theta(\log n)}$.

Isolamento da Variável k em Escala Base

Assumindo que nossa âncora de saída (Condição Base) processa arranjos iteracionais triviais de tamanho 1, inferimos $\mathbf{T(1) = 1}$.

Buscando convergência $n/2^k = 1$

$$\begin{aligned} \frac{n}{2^k} = 1 \quad &\Longrightarrow \quad n = 2^k \\ &\Longrightarrow \quad \log_2(n) = \log_2(2^k) \\ &\Longrightarrow \quad k = \log_2 n \end{aligned}$$

Retornando o valor k, finalizamos para as iterações da Pesquisa:

$$T(n) = T(1) + c \cdot (\log_2 n)$$

figs/logo.png

Conclusão Assintótica

O Algoritmo opera quantitativamente por custo marginal rigoroso $\mathbf{\Theta(\log n)}$.

Isolamento da Variável k em Escala Base

Assumindo que nossa âncora de saída (Condição Base) processa arranjos iteracionais triviais de tamanho 1, inferimos $\mathbf{T(1) = 1}$.

Buscando convergência $n/2^k = 1$

$$\begin{aligned} \frac{n}{2^k} = 1 \quad &\Longrightarrow \quad n = 2^k \\ &\Longrightarrow \quad \log_2(n) = \log_2(2^k) \\ &\Longrightarrow \quad k = \log_2 n \end{aligned}$$

Retornando o valor k, finalizamos para as iterações da Pesquisa:

$$T(n) = T(1) + c \cdot (\log_2 n)$$

figs/logo.png

Conclusão Assintótica

O Algoritmo opera quantitativamente por custo marginal rigoroso $\mathbf{\Theta(\log n)}$.

Isolamento da Variável k em Escala Base

Assumindo que nossa âncora de saída (Condição Base) processa arranjos iteracionais triviais de tamanho 1, inferimos $\mathbf{T(1) = 1}$.

Buscando convergência $n/2^k = 1$

$$\begin{aligned} \frac{n}{2^k} = 1 \quad &\Longrightarrow \quad n = 2^k \\ &\Longrightarrow \quad \log_2(n) = \log_2(2^k) \\ &\Longrightarrow \quad k = \log_2 n \end{aligned}$$

Retornando o valor k, finalizamos para as iterações da Pesquisa:

$$T(n) = T(1) + c \cdot (\log_2 n)$$

figs/logo.png

Conclusão Assintótica

O Algoritmo opera quantitativamente por custo marginal rigoroso $\mathbf{\Theta(\log n)}$.

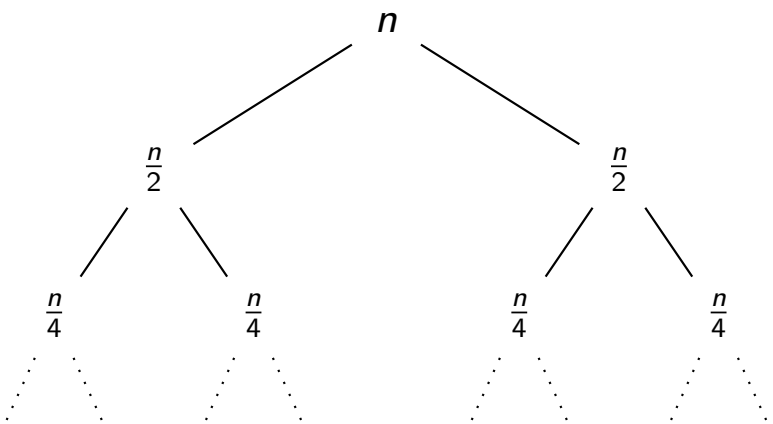
Projeção Fractal: Árvore de Recursão Clássica

Ferramenta visualmente proeminente aplicável aos *design* de algoritmos Divisão-e-Conquista modelados na essência $\mathbf{T(n) = 2T(n/2) + \Theta(n)}$.

figs/logo.png

Projeção Fractal: Árvore de Recursão Clássica

Ferramenta visualmente proeminente aplicável aos *design* de algoritmos Divisão-e-Conquista modelados na essência $\mathbf{T(n) = 2T(n/2) + \Theta(n)}$.



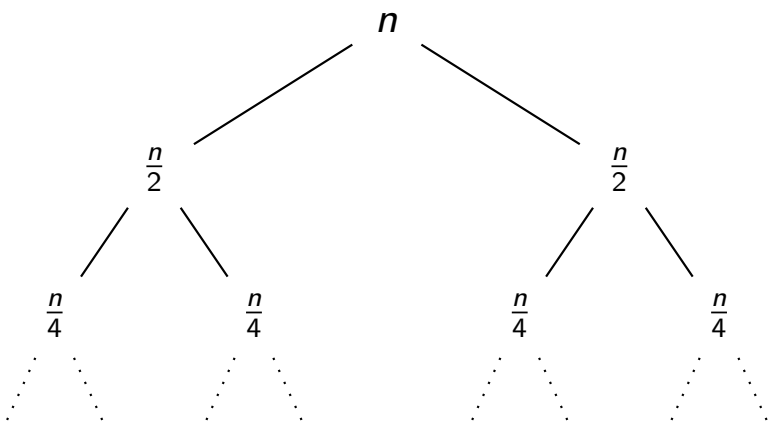
Custo: n

Somatório cumulativo $\sum_1^h n = n \times h$ onde a profundidade foliar $h = \log_2 n$.
Solução de Complexidade: $\mathbf{\Theta(n \log n)}$.

figs/logo.png

Projeção Fractal: Árvore de Recursão Clássica

Ferramenta visualmente proeminente aplicável aos *design* de algoritmos Divisão-e-Conquista modelados na essência $\mathbf{T(n) = 2T(n/2) + \Theta(n)}$.



Custo: n

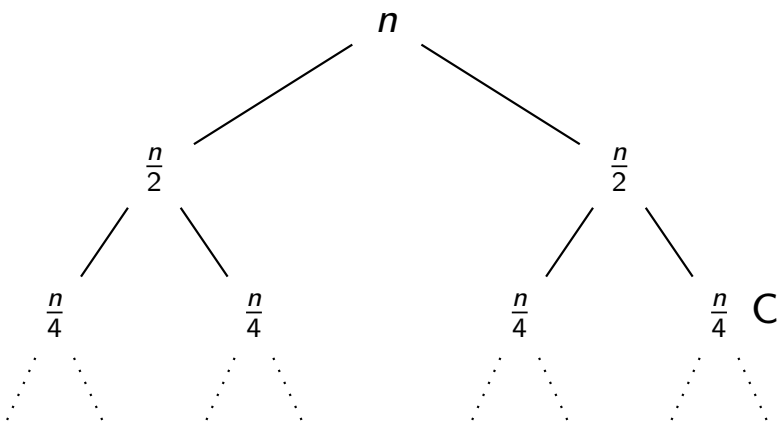
Custo: $\frac{n}{2} + \frac{n}{2} = n$

Somatório cumulativo $\sum_1^h n = n \times h$ onde a profundidade foliar $h = \log_2 n$.
Solução de Complexidade: $\mathbf{\Theta(n \log n)}$.

figs/logo.png

Projeção Fractal: Árvore de Recursão Clássica

Ferramenta visualmente proeminente aplicável aos *design* de algoritmos Divisão-e-Conquista modelados na essência $\mathbf{T(n) = 2T(n/2) + \Theta(n)}$.



Custo: n

Custo: $\frac{n}{2} + \frac{n}{2} = n$

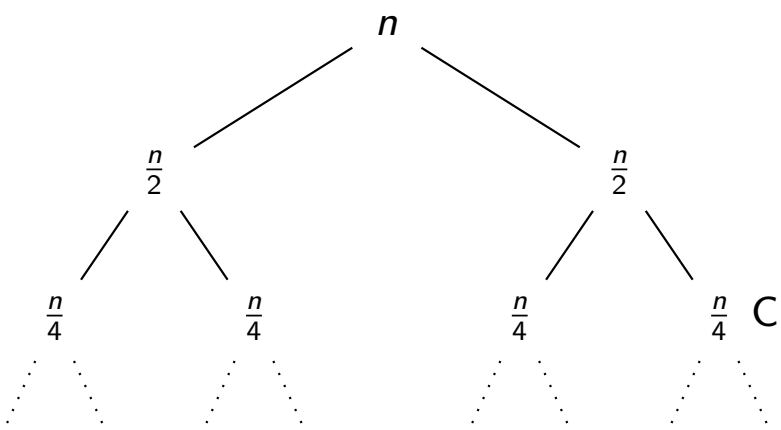
Custo: $4 \times \frac{n}{4} = n$

Somatório cumulativo $\sum_1^h n = n \times h$ onde a profundidade foliar $h = \log_2 n$.
Solução de Complexidade: $\mathbf{\Theta(n \log n)}$.

figs/logo.png

Projeção Fractal: Árvore de Recursão Clássica

Ferramenta visualmente proeminente aplicável aos *design* de algoritmos Divisão-e-Conquista modelados na essência $\mathbf{T(n) = 2T(n/2) + \Theta(n)}$.



Custo: n

Custo: $\frac{n}{2} + \frac{n}{2} = n$

Custo: $4 \times \frac{n}{4} = n$

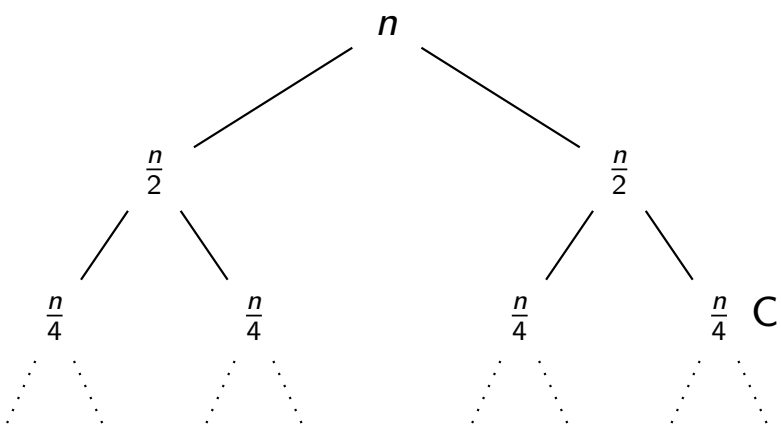
Nível Secundário...

Somatório cumulativo $\sum_1^h n = n \times h$ onde a profundidade foliar $h = \log_2 n$.
Solução de Complexidade: $\mathbf{\Theta(n \log n)}$.

figs/logo.png

Projeção Fractal: Árvore de Recursão Clássica

Ferramenta visualmente proeminente aplicável aos *design* de algoritmos Divisão-e-Conquista modelados na essência $\mathbf{T(n) = 2T(n/2) + \Theta(n)}$.



Custo: n

Custo: $\frac{n}{2} + \frac{n}{2} = n$

Custo: $4 \times \frac{n}{4} = n$

Nível Secundário...

Somatório cumulativo $\sum_1^h n = n \times h$ onde a profundidade foliar $h = \log_2 n$.
Solução de Complexidade: $\mathbf{\Theta(n \log n)}$.

figs/logo.png

Ensaio de Master: Formulário Direto Mapeado

Trata-se do *Cookbook* algorítmico global [?] visando solucionar eficientemente recorrências regidas pelo esqueleto genérico base:

Estrutura do Teorema

$$\mathbf{T(n) = aT\left(\frac{n}{b}\right) + f(n)} \quad \text{em que } a \geq 1, b > 1 \text{ ctes, e } f(n) \geq 0$$

Decifrando Parâmetros da Lógica Analítica:

- a: Subproblemas fracionados disparados ativamente recursão paralela na máquina.
- b: Fator de cisão/fatiamento reduzindo o problema nuclear.
- $f(n)$: Fator de carga de custo para repartição central e fusão/mesclagem subsequente do processo.

figs/logo.png

Ensaio de Master: Formulário Direto Mapeado

Trata-se do *Cookbook* algorítmico global [**?**] visando solucionar eficientemente recorrências regidas pelo esqueleto genérico base:

Estrutura do Teorema

$$\mathbf{T(n) = aT\left(\frac{n}{b}\right) + f(n)} \quad \text{em que } a \geq 1, b > 1 \text{ ctes, e } f(n) \geq 0$$

Decifrando Parâmetros da Lógica Analítica:

- a: Subproblemas fracionados disparados ativamente recursão paralela na máquina.
- b: Fator de cisão/fatiamento reduzindo o problema nuclear.
- $f(n)$: Fator de carga de custo para repartição central e fusão/mesclagem subsequente do processo.

figs/logo.png

Avaliação de Domínio: Comparativo Polinomial

O pilar metodológico se encarrega de comparar assintoticamente o esforço polinomial fixo $f(n)$ contra a expansão quantizada $n^{\log_b a}$ (volume iterativo da árvore tracionada).

Caso 1: Folhas predominam Se $f(n) = O(n^{\log_b a - \epsilon})$ com $\epsilon > 0 \implies \mathbf{T(n) = \Theta(n^{\log_b a})}$.
(A recursão cresce velozmente, devorando recursos marginais na borda foliar).

Caso 2: Empate de Forças Se $f(n) = \Theta(n^{\log_b a}) \implies \mathbf{T(n) = \Theta(n^{\log_b a} \cdot \log n)}$.
(Trabalho equitativo sendo distribuído ao longo dos níveis perfeitamente).

Caso 3: Raiz predominante Se $f(n) = \Omega(n^{\log_b a + \epsilon})$ para $\epsilon > 0$, dado a regularidade restrita $a \cdot f(\frac{n}{b}) \leq c \cdot f(n) \implies \mathbf{T(n) = \Theta(f(n))}$.

figs/logo.png

Avaliação de Domínio: Comparativo Polinomial

O pilar metodológico se encarrega de comparar assintoticamente o esforço polinomial fixo $f(n)$ contra a expansão quantizada $n^{\log_b a}$ (volume iterativo da árvore tracionada).

Caso 1: Folhas predominam Se $f(n) = O(n^{\log_b a - \epsilon})$ com $\epsilon > 0 \implies \mathbf{T(n) = \Theta(n^{\log_b a})}$.
(A recursão cresce velozmente, devorando recursos marginais na borda foliar).

Caso 2: Empate de Forças Se $f(n) = \Theta(n^{\log_b a}) \implies \mathbf{T(n) = \Theta(n^{\log_b a} \cdot \log n)}$.
(Trabalho equitativo sendo distribuído ao longo dos níveis perfeitamente).

Caso 3: Raiz predominante Se $f(n) = \Omega(n^{\log_b a + \epsilon})$ para $\epsilon > 0$, dado a regularidade restrita $a \cdot f(\frac{n}{b}) \leq c \cdot f(n) \implies \mathbf{T(n) = \Theta(f(n))}$.

figs/logo.png

Avaliação de Domínio: Comparativo Polinomial

O pilar metodológico se encarrega de comparar assintoticamente o esforço polinomial fixo $f(n)$ contra a expansão quantizada $n^{\log_b a}$ (volume iterativo da árvore tracionada).

Caso 1: Folhas predominam Se $f(n) = O(n^{\log_b a - \epsilon})$ com $\epsilon > 0 \implies \mathbf{T(n) = \Theta(n^{\log_b a})}$.
(A recursão cresce velozmente, devorando recursos marginais na borda foliar).

Caso 2: Empate de Forças Se $f(n) = \Theta(n^{\log_b a}) \implies \mathbf{T(n) = \Theta(n^{\log_b a} \cdot \log n)}$.
(Trabalho equitativo sendo distribuído ao longo dos níveis perfeitamente).

Caso 3: Raiz predominante Se $f(n) = \Omega(n^{\log_b a + \epsilon})$ para $\epsilon > 0$, dado a regularidade restrita $a \cdot f(\frac{n}{b}) \leq c \cdot f(n) \implies \mathbf{T(n) = \Theta(f(n))}$.

figs/logo.png

Avaliação de Domínio: Comparativo Polinomial

O pilar metodológico se encarrega de comparar assintoticamente o esforço polinomial fixo $f(n)$ contra a expansão quantizada $n^{\log_b a}$ (volume iterativo da árvore tracionada).

Caso 1: Folhas predominam Se $f(n) = O(n^{\log_b a - \epsilon})$ com $\epsilon > 0 \implies \mathbf{T(n) = \Theta(n^{\log_b a})}$.
(A recursão cresce velozmente, devorando recursos marginais na borda foliar).

Caso 2: Empate de Forças Se $f(n) = \Theta(n^{\log_b a}) \implies \mathbf{T(n) = \Theta(n^{\log_b a} \cdot \log n)}$.
(Trabalho equitativo sendo distribuído ao longo dos níveis perfeitamente).

Caso 3: Raiz predominante Se $f(n) = \Omega(n^{\log_b a + \epsilon})$ para $\epsilon > 0$, dado a regularidade restrita $a \cdot f(\frac{n}{b}) \leq c \cdot f(n) \implies \mathbf{T(n) = \Theta(f(n))}$.

figs/logo.png

Casos de Aplicação Direta Mestre

Embate Intelectual Clássico - Tente determinar o limite Assintótico:

Exemplo 1 (Raízes Duplas)

$T(n) = 4T(n/2) + n \implies n^{\log_2 4} = n^2$. Folhas sobrepujam n.
Resposta: **Caso 1** $\rightarrow \Theta(n^2)$.

Exemplo 2 (Esboço Singular)

$T(n) = T(2n/3) + 1 \implies b = 3/2 \implies n^{\log_{1.5} 1} = n^0 = 1$. Concorrem equilibrados
Resposta: **Caso 2** $\rightarrow \Theta(\log n)$.

Exemplo 3 (Carga Complexificada)

$T(n) = 9T(n/3) + n^3 \implies n^{\log_3 9} = n^2$. Raiz (composição) supera limite $n^3 > n^2$
Resposta: **Caso 3** $\rightarrow \Theta(n^3)$.

Casos de Aplicação Direta Mestre

Embate Intelectual Clássico - Tente determinar o limite Assintótico:

Exemplo 1 (Raízes Duplas)

$T(n) = 4T(n/2) + n \implies n^{\log_2 4} = n^2$. Folhas sobrepujam n.
Resposta: **Caso 1** $\rightarrow \Theta(n^2)$.

Exemplo 2 (Esboço Singular)

$T(n) = T(2n/3) + 1 \implies b = 3/2 \implies n^{\log_{1.5} 1} = n^0 = 1$. Concorrem equilibrados
Resposta: **Caso 2** $\rightarrow \Theta(\log n)$.

Exemplo 3 (Carga Complexificada)

$T(n) = 9T(n/3) + n^3 \implies n^{\log_3 9} = n^2$. Raiz (composição) supera limite $n^3 > n^2$
Resposta: **Caso 3** $\rightarrow \Theta(n^3)$.

/logo.png

Casos de Aplicação Direta Mestre

Embate Intelectual Clássico - Tente determinar o limite Assintótico:

Exemplo 1 (Raízes Duplas)

$T(n) = 4T(n/2) + n \implies n^{\log_2 4} = n^2$. Folhas sobrepujam n.
Resposta: **Caso 1** $\rightarrow \Theta(n^2)$.

Exemplo 2 (Esboço Singular)

$T(n) = T(2n/3) + 1 \implies b = 3/2 \implies n^{\log_{1.5} 1} = n^0 = 1$. Concorrem equilibrados
Resposta: **Caso 2** $\rightarrow \Theta(\log n)$.

Exemplo 3 (Carga Complexificada)

$T(n) = 9T(n/3) + n^3 \implies n^{\log_3 9} = n^2$. Raiz (composição) supera limite $n^3 > n^2$
Resposta: **Caso 3** $\rightarrow \Theta(n^3)$.

/logo.png

Casos de Aplicação Direta Mestre

Embate Intelectual Clássico - Tente determinar o limite Assintótico:

Exemplo 1 (Raízes Duplas)

$T(n) = 4T(n/2) + n \implies n^{\log_2 4} = n^2$. Folhas sobrepujam n.
Resposta: **Caso 1** $\rightarrow \Theta(n^2)$.

Exemplo 2 (Esboço Singular)

$T(n) = T(2n/3) + 1 \implies b = 3/2 \implies n^{\log_{1.5} 1} = n^0 = 1$. Concorrem equilibrados
Resposta: **Caso 2** $\rightarrow \Theta(\log n)$.

Exemplo 3 (Carga Complexificada)

$T(n) = 9T(n/3) + n^3 \implies n^{\log_3 9} = n^2$. Raiz (composição) supera limite $n^3 > n^2$
Resposta: **Caso 3** $\rightarrow \Theta(n^3)$.

/logo.png

Referências I

- Lee K.D., Hubbard S. (2015) Trees. In: Data Structures and Algorithms with Python. Undergraduate Topics in Computer Science. Springer, Cham. https://doi.org/10.1007/978-3-319-13072-9_6
- Hubbard, J. (2007). Schaum's Outline of Data Structures with Java. http://www.amazon.com/Schaums-Outline-Data-Structures-Java/dp/0071476989
- Cormen, T. H., Leiserson, C. E., & Stein, R. L. R. (2012). Algoritmos: teoria e prática. https://books.google.com.br/books?id=6iA4LgEACAAJ.
- Ascenio, Ana Fernanda Gomes. Estrutura de dados: Algoritmos, análise da complexidade e implementações em Java e C++. São Paulo: Pearson Prentice Hall, 2010.

figs/logo.png

Referências II

Szwarcfiter, Jayme Luiz. Estruturas de dados e seus algoritmos / Jayme Luiz Szwarcfiter, Lilian Markenzon. 3.ed. [Reimpr.]. - Rio de Janeiro : LTC, 2015.

Capítulo 20: Árvores — documentação Aprenda Computação com Python 3.0 2009.1. https://mange.ifrn.edu.br/python/aprenda-com-py3/capitulo_20.html

figs/logo.png

figs/logo.png

Recursão

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br

[1]Faculdade de Sistemas de Informação - UFPA/CUTINS

25 de março de 2026